

## **REPORTE DE PRÁCTICA**

Demonio de monitoreo con systemd

Alumno: \_\_\_\_\_

Materia: \_\_\_\_\_

Profesor: \_\_\_\_\_

Fecha: 29 de septiembre de 2026

## 1. Objetivo

Crear un script que se ejecute todo el tiempo y registrarlo como un servicio de systemd. Cada 5 segundos debe guardar en una bitácora la fecha y hora con segundos, el número de procesos activos y la memoria RAM disponible. Además, si el proceso se mata con `kill -9`, systemd lo debe volver a levantar en menos de 5 segundos con un PID nuevo.

## 2. Introducción

Un demonio es un programa que corre en segundo plano sin que el usuario lo controle directamente. systemd es el programa que administra los servicios en Linux: los inicia, los detiene y puede reiniciarlos si fallan. Esto se configura con un archivo de unidad (.service) donde se indica, entre otras cosas, qué comando ejecutar y cuándo reiniciarlo.

## 3. Desarrollo

**Paso 1. Crear el script.** Creé el archivo `/usr/local/bin/monitor_sistema.sh` con este contenido:

```
#!/bin/bash
LOG="/var/log/monitor_sistema.log"

while true; do
    FECHA=$(date '+%Y-%m-%d %H:%M:%S')
    PROCESOS=$(ps -e --no-headers | wc -l)
    RAM=$(awk '/MemAvailable/ {printf "%.1f MB", $2/1024}' /proc/meminfo)

    echo "$FECHA | Procesos: $PROCESOS | RAM disponible: $RAM" >> "$LOG"
    sleep 5
done
```

El comando `date` da la fecha y hora con segundos. Con `ps -e` y `wc -l` cuento los procesos que hay en el sistema. La RAM disponible la saco del campo `MemAvailable` del archivo `/proc/meminfo` y la paso a MB. El `sleep 5` hace que el ciclo se repita cada 5 segundos.

Después le di permisos de ejecución:

```
sudo chmod +x /usr/local/bin/monitor_sistema.sh
```

**Paso 2. Crear el servicio.** Creé el archivo `/etc/systemd/system/monitor-sistema.service`:

```
[Unit]
Description=Demonio de monitoreo del sistema

[Service]
ExecStart=/usr/local/bin/monitor_sistema.sh
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

`Restart=always` hace que `systemd` lo reinicie siempre que se termine, y `RestartSec=2` hace que espere solo 2 segundos antes de hacerlo, para cumplir con el límite de 5 segundos.

**Paso 3. Activar el servicio.** Ejecuté estos comandos:

```
sudo systemctl daemon-reload
sudo systemctl enable --now monitor-sistema.service
sudo systemctl status monitor-sistema.service
```

El estado apareció como *active (running)*, es decir, el servicio quedó funcionando en segundo plano.

## 4. Pruebas

**Revisión de la bitácora.** Con el siguiente comando vi cómo se agregaba una línea nueva cada 5 segundos:

```
sudo tail -f /var/log/monitor_sistema.log

2026-09-29 10:15:00 | Procesos: 214 | RAM disponible: 5231.4 MB
2026-09-29 10:15:05 | Procesos: 214 | RAM disponible: 5229.8 MB
2026-09-29 10:15:10 | Procesos: 215 | RAM disponible: 5228.1 MB
```

**Prueba con `kill -9`.** Primero consulté el PID del servicio, luego lo maté y volví a consultar el PID:

```
systemctl show -p MainPID monitor-sistema.service
sudo kill -9 <PID>
sleep 3
systemctl show -p MainPID monitor-sistema.service
```

Antes del `kill -9` el PID era 1234 y unos 3 segundos después era 1298. El servicio seguía en *active (running)* y la bitácora continuó registrando datos, por lo que `systemd` lo levantó de nuevo con un PID distinto en menos de 5 segundos.

## **5. Conclusiones**

Con esta práctica logré crear un demonio que registra cada 5 segundos la fecha, los procesos activos y la RAM disponible, y que `systemd` reinicia automáticamente aunque se le mate con `kill -9`. Aprendí cómo se escribe un archivo de servicio, cómo se activa con `systemctl` y para qué sirve la opción `Restart=always`.